E.1.1 Pitfalls

Recursion is not free of risks.

First, we need to make sure to define a base case that is *always* possible to reach. In the example above, if we forgot to check that the argument `n` is positive, `fibonacci(-1)` would go through every single integer number that can be stored in *JavaScript* before hitting the base case for `n === 0`. But more likely, it would throw a `RangeError: Maximum call stack size exceeded` long before getting anywhere near the base case.

Of course, even using a loop you'd need to get stopping conditions right, otherwise you might end up with an infinite loop.

And even with that extra check, we are not safe yet. In weakly-typed languages like JavaScript, code-coercion will try to convert arguments passed to our function to numbers. Try to call `fibonacci('a')` or `fibonacci(true)` instead: the method above doesn't check that n has the proper type, `'a' - 1` returns `NaN`, as well as `NaN - 1`. The result is a (theoretically) infinite recursion call stack, that ends up once again in a `RangeError`.

Another possible downside is waste of resources. If you look closely at the code above, you can understand why this is not the best use case for recursion. For instance, try track down the recursive calls we make when we call `fibonacci(4)`: